



Как просить, *чтобы получалось.*

Хороший промт похож на ТЗ для стажёра: цель, ограничения, формат, иногда пример. На этом этапе мы учимся ставить задачи, итерировать, откатываться, генерировать варианты — и понимать, что Claude вернул.

ДЛИТЕЛЬНОСТЬ

4–6 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

структуру промпта · итерация · откат · варианты · чтение результата

ГЛАВНЫЙ ИТОГ

первая версия от Claude уже близка к нужной

АРТЕФАКТ

калькулятор сложного процента, игра «угадай число», + ваши вариации

На этом этапе ничего нового устанавливать не нужно — всё, что мы собрали в этапе 00, продолжает работать. Один skill будет полезен, но опционально.

ОБЯЗАТЕЛЬНО

Из этапа 00

- **Node.js, Claude Code, VS Code** — проверьте: в терминале `claude --version` должно вывести номер версии, а не «command not found».
- Аккаунт Anthropic, в который вы вошли в этапе 00.

ОПЦИОНАЛЬНО

Skill `superpowers:brainstorming`

Что это: skill, который *учит* Claude вести с вами короткий брейншторм перед задачей — задавать уточняющие вопросы, прежде чем что-то делать.

Зачем сейчас: на первых трёх-четырёх задачах этапа полезно, чтобы Claude задавал «правильные вопросы» — так мы учимся видеть, чего не хватало в вашем промпте.

Что встанет: пакет `superpowers` целиком — внутри несколько skills (`brainstorming`, `writing-plans`, `executing-plans`, `verification-before-completion` и другие). Сейчас нам нужен только `brainstorming`; остальные пригодятся в этапах 04 и 06.

```
# внутри claude (slash-команда, не shell)
/plugin install superpowers@claude-plugins-official
```

Где взять: встроенный маркетплейс Claude Code (`claude-plugins-official`); исходники — github.com/obra/superpowers.

Проверить установку: запустите `claude`, попросите «сделай мне сайт» — Claude должен сначала задать уточняющие вопросы, а не броситься писать код.

Если skill не установили – не страшно, все задачи этапа решаются и без него. Но при первой возможности поставьте: брейншторм-привычка очень помогает на этапах 04 и 06.

Анатомия хорошего промпта: цель / ограничения / формат / пример

Сравните три способа попросить одно и то же:

Плохо:

```
> сделай калькулятор
```

Чуть лучше:

```
> сделай калькулятор сложного процента в html
```

Хорошо:

```
> сделай калькулятор сложного процента — одностраничник index.html.  
> ввод: сумма, ставка %/год, срок в годах.  
> вывод: итоговая сумма + таблица по годам + график.  
> стиль — минималистичный, тёплая бумажная палитра, шрифт засечковый.  
> одним файлом, без зависимостей, чтобы открывалось двойным кликом.
```

В хорошем промпте есть **четыре слоя**:

1. **Цель** — что должно получиться. «*калькулятор сложного процента*».
2. **Ограничения** — чего не делать или какими средствами. «*одним файлом, без зависимостей*».
3. **Формат** — в каком виде получить результат. «*одностраничник index.html*», «*ввод X, вывод Y*».
4. **Пример или стиль** — на что похоже. «*минималистичный, тёплая бумажная палитра*».

Чем больше ясности в этих четырёх слоях, тем точнее первая версия. Но *не* переусердствуйте: если каждый раз писать промпт на десять строк, вы потеряете в скорости. На простые задачи — короткий промпт.

Когда давать длинный контекст, когда короткий

«Контекст» — это вся информация, которую Claude видит в текущем диалоге: ваши промпты, прикреплённые файлы, прошлые ответы. Чем больше контекста, тем больше Claude знает — но и тем дороже стоит каждый ответ.

Длинный контекст полезен, когда:

- задача завязана на конкретные файлы/данные;
- есть нетривиальные правила или стиль;
- важна совместимость с уже сделанным.

Короткий контекст лучше, когда:

- задача типовая;
- вы не уверены, чего хотите — быстрее показать первую версию и уточнить;
- вы только начинаете сессию.

Хорошее правило: *сначала короткий промпт, потом доуточняем по ходу*. Это быстрее, чем сразу писать большой.

Итерация: «почти, но...», «нет, не так», «теперь добавь»

Главный навык всего курса: *не писать «идеальный» промпт сразу, а поправлять короткими шагами*.

Полезные фразы для итерации:

- «почти, но шрифт слишком крупный»;
- «нет, я имел в виду...»;
- «сделай ту же страницу, но в тёмной теме»;
- «добавь кнопку «сбросить» рядом с «посчитать»»;
- «сохрани текущую версию как v1.html и сделай v2.html, где...».

Claude быстрый. Пять-шесть итераций по 30 секунд работают лучше, чем десять минут шлифовки одного промпта.

Откат: «верни как было», «забудь последнее», Esc

Если итерация увела куда-то не туда, нужно уметь возвращаться:

- **Esc** — во время выполнения. Останавливает текущее действие, ничего не трогает.
- **«верни файл X как был до последнего изменения»** — Claude использует git или внутренние backup.
- **«забудь то, что я только что сказал, начнём заново с вот этого...»** — явно говорите, что прошлый промпт «не считается». Claude уберёт его из своих рассуждений.
- **«удали файл X»** — самый прямой откат, если нужно начать с нуля.

Привыкайте откатываться без стеснения — это часть нормальной работы с агентом, а не признак ошибки.

Генерация вариантов: «сделай ещё две версии»

Один из любимых приёмов: попросить Claude сделать *несколько вариантов одной задачи*. Полезно, когда вы не уверены, какой стиль/подход хотите.

```
> сохрани текущую версию как v1-classic.html.  
> теперь сделай v2-minimal.html (более минималистичную)  
> и v3-playful.html (с яркими цветами и анимацией).  
> одни и те же данные, разный стиль.
```

На выходе — три файла. Открываете все три, сравниваете в браузере, выбираете любимый или просите что-то соединить.

Этот приём незаменим, когда:

- вы ищете «свой» стиль и ещё не знаете его;
- нужно показать заказчику/другу выбор;

- хотите быстро увидеть, какие подходы Claude вообще считает уместными.

Чтение результата: что Claude сделал, что предположил, что пропустил

После каждого ответа Claude в терминале есть *две части*:

1. **Что он сделал** — список изменений (созданные/изменённые файлы, выполненные команды). Это видно сразу.
2. **Что он предположил** — решения, которые он принял за вас.
Например: «использовал библиотеку Chart.js», «выбрал тёмную тему», «считал ставку годовой».

Привыкайте читать *оба слоя*. Если Claude предположил что-то не так — на следующей итерации поправьте: *«нет, ставка должна быть месячной»*.

Если что-то *пропустил* — явно укажите: *«ты не добавил кнопку сброса, добавь»*.

Это и есть «навык чтения результата» — один из ключевых для всего курса.

Сохранение и возобновление сессий: `--continue`, `--resume`

Сессия Claude Code не пропадает, когда вы закрываете терминал. Можно вернуться позже:

```
# продолжить последнюю сессию в текущей папке
$ claude --continue

# выбрать одну из недавних сессий из списка
$ claude --resume
```

Полезно, когда:

- задача большая, вы вернулись на следующий день;
- не хочется заново объяснять контекст;
- был сбой и диалог закрылся неожиданно.

На этапе 04 разберём, как длинные сессии «уплотнять» через `/compact`, чтобы Claude не тонул в старых сообщениях.

Плохой промпт vs хороший · 5 пар на разбор

Прочитайте каждую пару и попробуйте сами сформулировать «хороший вариант» для своей задачи — например, для карточки фильма из этапа 00. Дальше будет прогон, где надо применить в диалоге.

PRACTICUM

Пять пар «плохой → хороший»

1. Сайт

```
# плохо
> сделай мне сайт

# хорошо
> сделай одностраничник про мою кофейню «Утро».
> вверху имя и слоган, ниже меню (3 категории по 4 позиции),
> время работы, карта (просто адрес курсивом), футер с инстаграмом.
> стиль — как в скандинавском кафе, тёплые бежевые тона.
```

2. Калькулятор

```
# плохо
> калькулятор

# хорошо
> калькулятор калорий за день: ввод — продукты с весом,
> результат — общая калорийность + БЖУ.
> одна страница html, без зависимостей, простая таблица.
```

3. Игра

```
# плохо
> сделай игру
# хорошо
> сделай браузерную игру «угадай число от 1 до 100».
> компьютер загадывает, я угадываю, он подсказывает «больше/меньше».
> 7 попыток, в конце — статистика.
> одна html-страница, тёмная тема, крупные кнопки.
```

4. Анализ

```
# плохо
> проанализируй мои траты
# хорошо
> в файле expenses.csv — мои траты за квартал (дата, сумма, категория, заметка).
> собери short-отчёт в expenses-report.md:
> 1) общая сумма по месяцам;
> 2) топ-3 категорий;
> 3) две неожиданные заметки, которые требуют моего внимания.
> не давай советов «как экономить» — только факты.
```

5. Дизайн

```
# плохо
> сделай страницу красивой
# хорошо
> перепиши index.html в стиле японской газеты shimbun:
> засечковый шрифт, два столбца, ornate horizontal rules,
> никаких анимаций, бумажный фон.
```

Что общего у «хороших»? **Конкретное имя файла или сущности, измеримые ограничения** («3 категории по 4 позиции», «7 попыток»), **намёк на стиль** через узнаваемое сравнение («скандинавское кафе», «японская газета shimbun»).

PRACTICUM

Прогон · Одна страница, пять итераций

1. В папке `claude-course` создайте новую папку `iter`, запустите там `claude`.

2. **Промпт 1.** «сделай простую страницу-таймер на 15 минут. кнопка старт/стоп, цифры крупным шрифтом, центр экрана».
3. Откройте файл в браузере. Поправьте **2:** «поменяй цвет цифр на тёплый оранжевый».
4. Поправьте **3:** «добавь кнопку «сброс» рядом со стартом, тех же размеров».
5. Поправьте **4:** «сохрани текущую версию как *timer-classic.html*. сделай ещё *timer-noir.html* — чёрный фон, неоновые цифры».
6. Поправьте **5:** «в *noir* добавь тиканье каждую секунду через `<audio>` (синтезируй через *web-audio*, без скачивания)».
7. В конце сделайте промпт-«ревизию»: «покажи в виде списка, что ты предположил за меня, чего я не указал».

Что мы тренируем: **работать через короткие шаги, генерировать варианты, видеть собственные пробелы в промптах** через ревизию-вопрос.

Эти два проекта — «база» курса. На них учимся писать промпты, итерировать, генерировать варианты. К концу у вас два отдельных артефакта.

Калькулятор сложного процента

Цель: одностраничник в браузере. Ввод — сумма, ставка, срок. Вывод — итог по годам в таблице и графиком.

Шаги

1. Создайте папку `compound` в `claude-course`, запустите `claude`.
2. Начальный промпт (используем все четыре слоя — цель, ограничения, формат, стиль):

```
> сделай калькулятор сложного процента, одностраничник index.html.  
> ввод: сумма (число), ставка %/год, срок в годах.  
> вывод: итоговая сумма + таблица «год | вклад на конец года»  
> + график рост капитала (можно chart.js через CDN).  
> стиль — минималистичный, тёплая бумажная палитра,  
> засечковый шрифт, центр страницы.  
> одним файлом, открывается двойным кликом.
```

3. Откройте `index.html` двойным кликом. Прогоните 1000 Р, 10%, 10 лет — должно быть около 2594.
4. Уточняйте по ходу:
 - «сделай ввод тремя полями в одну строку, не столбиком»;
 - «добавь поле «частота начислений» с опциями: годовая, квартальная, месячная»;
 - «в таблицу добавь столбец «прирост за год»»;
 - «добавь кнопку «сбросить»».
5. Когда нравится — попросите три варианта: «сохрани как `compound-classic.html`. сделай ещё `compound-dark.html` (тёмная тема) и `compound-print.html` (как страница финансовой газеты, чёрно-белый)».
6. Финальный аудит: «перечисли в виде списка, что ты принял за меня — цвета, шрифты, валюту, формулу».

Что вы здесь освоили

- четырёх-слойный промпт на нетривиальной задаче;

- итеративные уточнения вокруг одной страницы;
- генерацию нескольких стилизованных вариантов;
- аудит-вопрос «что ты принял за меня».

Игра «угадай число»

Цель: браузерная игра. Компьютер загадывает число от 1 до 100, у вас 7 попыток, после каждой — «больше / меньше». Рекорды сохраняются в файл рядом.

Шаги

1. Папка `guess`, `claude` в ней.

2. Начальный промпт:

```
> сделай игру «угадай число» в браузере. одна html-страница.  
> компьютер загадывает целое число от 1 до 100.  
> у меня 7 попыток. ввожу число, нажимаю «угадать»,  
> компьютер отвечает «больше» / «меньше» / «угадал».  
> в конце игры — статистика (попыток, время) и кнопка «ещё раз».  
> рекорды (имя + попыток + время) сохраняй в localStorage.  
> стиль — крупная типографика, центр экрана, тёплая палитра.
```

3. Поиграйте раз пять. Заметьте, что Claude мог что-то «не то» сделать — например, не показывать сколько попыток осталось.

4. Уточняйте:

- «добавь крупный счётчик «осталось *N* попыток» сверху»;
- «при проигрыше показывай загаданное число»;
- «сделай таблицу рекордов прокручиваемой, не шире контента»;
- «добавь три уровня сложности: 1–50 / 1–100 / 1–500».

5. Сохраните как `guess-v1.html`. Сделайте «вариант для детей»:

```
> сделай guess-kids.html: тот же геймплей, но  
> крупные эмодзи вместо «больше / меньше» (👉 / 👈),  
> озвучка победы через web-audio (без файлов),  
> диапазон 1–20, 5 попыток.
```

6. Аудит: «перечисли, что ты добавил без явной просьбы — например, шрифты, иконки, тексты».

Что вы здесь освоили

- задача с состоянием (счётчик, рекорды, режимы);
- превратить базовую версию в «детскую» одной просьбой;
- аудит «что Claude добавил по своей инициативе».

Эти задачи — вариации проектов А и В. Структура та же — четырёх-
слойный промпт, 4–6 итераций, 2–3 варианта стиля, аудит. Когда что-то
идёт не так — вернитесь к инструкциям выше.

Калькулятор накопления на цель

Цель: одностраничник, который отвечает на вопрос «сколько копить в месяц, чтобы накопить N к дате T при ставке X%». Те же четыре слоя, что в проекте А.

Подсказки

- Структура та же, что у проекта А — меняется формула и имена полей.
- Поля ввода: цель Р, дата цели (или «через X лет»), текущая сумма, ставка %/год.
- Вывод: «нужно откладывать N Р/мес» + таблица по месяцам + график.
- Сделайте 2–3 варианта стиля, как мы делали с compound.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть три файла-версии: `save-classic.html`, `save-dark.html`, `save-print.html`;
- ✓ правильно считает: при цели 1 000 000, ставке 8% и сроке 10 лет нужно откладывать примерно 5–6 тысяч в месяц (формула аннуитета);
- ✓ есть кнопка «сбросить» и понятная подпись валюты;
- ✓ на узком окне (телефон) ввод и таблица не ломаются;
- ✓ вы провели хотя бы один аудит-промпт «что ты принял за меня» и поправили хотя бы одно из предположений.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Сравни три моих файла `save-*.html`. Какой из них покажет лучший результат финансовому консультанту? Где формула считается одинаково? Найди расхождения, если есть». Это вторая часть аудита — сверка цифр, не только дизайна.

Игра «камень-ножницы-бумага»

Цель: браузерная игра против компьютера, серия из 5 раундов, счёт сохраняется в `localStorage`. Та же структура, что у «угадай число».

Подсказки

- Три кнопки выбора (камень / ножницы / бумага) — крупными иконками или эмодзи.
- В конце раунда показывайте, что выбрал компьютер.
- Счёт «вы X : Y компьютер» сверху страницы.
- После 5 раундов — «победа / ничья / поражение» крупно, кнопка «ещё».
- Сделайте «детскую» версию с анимацией кулака на «1, 2, 3».

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ две версии: `rps-v1.html` (классика) и `rps-kids.html` (детская с анимацией);
- ✓ счёт сохраняется между перезагрузками браузера;
- ✓ выбор компьютера действительно случайный (сыграйте 10 раз и убедитесь, что не всегда «бумага»);
- ✓ после 5 раундов есть финальный экран и возможность сыграть ещё;
- ✓ на телефоне кнопки крупные, удобно жать пальцем.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Сыграй сам в `rps-v1.html`, открыв его в браузере через `playwright`. Сделай 20 ходов «камень». Распределение ответов компьютера должно быть примерно равномерным — проверь и скажи, нет ли перекоса».

Этап 01 расширяет понятия из этапа 00 и добавляет несколько новых. В тексте они подчёркнуты пунктиром — наведите/коснитесь, и всплывёт справка.

Промпт	<i>(расширено)</i> Формулировка задачи. Хороший промпт состоит из четырёх слоёв: цель, ограничения, формат, пример / стиль . На простые задачи — короткий промпт, на сложные — все четыре слоя.
Контекст	Вся информация, которую Claude учитывает в текущем диалоге: ваши промпты, прикреплённые файлы, прошлые ответы. Чем длиннее контекст — тем больше Claude знает, тем дороже стоит каждый ответ. На этапе 04 разбираем, как им управлять.
Итерация	<i>(повтор)</i> Повторный заход по той же задаче с поправкой. Главный навык курса — не писать «идеальный» промпт сразу, а поправлять короткими шагами.
Откат	Команда вернуть Claude или файл к предыдущему состоянию. Способы: Esc (во время выполнения), «верни как было», «забудь предыдущее», «удали этот файл».
Варианты	Несколько разных версий одной задачи, сгенерированных одним запросом. «сохрани как v1, теперь сделай v2 в другом стиле». Полезно, когда не уверены, какой подход хотите.
Сессия	Один диалог с Claude от запуска до выхода. Сессия не пропадает при закрытии терминала — продолжите через <code>claude --continue</code> или выберите из списка через <code>claude --resume</code> .
Аудит-промт	Финальный вопрос диалога: «перечисли, что ты принял за меня — цвета, формулу, валюту, шрифты». Помогает увидеть «слепые пятна» вашего промпта и сделать их явными в следующий раз.
CDN	Сервер, с которого браузер скачивает готовую библиотеку (например, <code>chart.js</code>) при открытии страницы. В промпте «через CDN» означает: не качай и не ставь — вставь ссылку, и всё.
localStorage	«Память» браузера для одной страницы — данные сохраняются между перезагрузками. Используем для рекордов, сохранённых тем, состояния игр.
brainstorming	Skill, который <i>учит</i> Claude задавать уточняющие вопросы перед началом задачи. Полезен, когда вы только начинаете и не всегда сами знаете, чего хотите. Установлен опционально в § 0.

Семь вопросов на закрепление

Постарайтесь сначала ответить мысленно или вслух — только потом раскрывайте ответ. После сравнения отметьте: *знал* или *не знал*. По итогам подскажу, что повторить.

01 Из каких четырёх слоёв состоит хороший промпт?

Цель (что должно получиться) · **ограничения** (чего не делать / какими средствами) · **формат** (в каком виде получить результат) · **пример / стиль** (на что похоже).

02 Что лучше: написать «идеальный» десяти-строчный промпт сразу или начать с короткого и поправлять? Почему?

Лучше короткий + итерации. Claude быстрый: 5–6 поправок по 30 секунд работают точнее и быстрее, чем 10 минут шлифовки одного промпта. Кроме того, вы увидите, что Claude предположил — это сразу подсветит ваши собственные пробелы.

03 Чем отличается «откат» через `Esc` от «верни файл как был»?

`Esc` — останавливает *текущее* действие во время выполнения, ничего не трогает. «верни файл как был» — откатывает *уже сделанные* изменения через git/backup. Первое — «не делай вот это», второе — «отмени то, что сделал».

04 Зачем просить Claude сгенерировать *несколько вариантов* одной страницы? Когда это особенно полезно?

Когда вы не уверены, какой стиль или подход хотите. Сравнить три открытых вкладки в браузере проще, чем описывать стиль словами. Также полезно показать варианты заказчику или другу — и сразу увидеть, какие подходы Claude вообще считает уместными.

05 Что такое «аудит-пром프트» и зачем он нужен?

Финальный вопрос: «*перечисли, что ты принял за меня — цвета, формулу, валюту, шрифты*». Подсвечивает «слепые пятна» промпта — места, где Claude сделал выбор за вас. В следующий раз сможете указать эти параметры явно или сознательно отдать на его выбор.

06 Как продолжить вчерашнюю сессию Claude Code в той же папке, не объясняя контекст заново?

`claude --continue` — продолжит последнюю сессию в текущей папке. `claude --resume` — покажет список недавних сессий, можно выбрать любую.

07 Промпт «*сделай страницу красивой*» — почему это плохо? Как переписать?

«Красиво» — абстрактно, у каждого своё определение. Claude выберет «средне-ИИ-усреднённый» вариант, который вряд ли совпадёт с вашим. Лучше: дать *узнаваемое сравнение* (стиль японской газеты, скандинавское кафе, brutalism, Bauhaus) и конкретные параметры (шрифт, палитра, ширина, отступы).

Что я теперь умею

Отметьте уверенно освоенное. Сомневаетесь — вернитесь к соответствующей теме. Этап готов закрыть, когда все галочки стоят.

- ☐ Знаю четыре слоя промпта: цель, ограничения, формат, пример.
 - ☐ Уверенно работаю короткими итерациями: «почти, но...», «нет, не так», «теперь добавь».
 - ☐ Знаю три способа отката: `Esc`, «верни как было», «забудь предыдущее».
 - ☐ Прошу несколько стилевых вариантов одной задачи и сравниваю их.
 - ☐ Делаю аудит-промпт «что ты принял за меня» и поправляю предположения.
 - ☐ Знаю, как возобновить сессию через `claude --continue` и `claude --resume`.
 - ☐ Сделал четыре проекта: калькулятор сложного процента, игру «угадай число», калькулятор накопления на цель, «камень-ножницы-бумага». Все работают двойным кликом.
-
-

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

*«Промпт-инжиниринг — это не навык
писать длинные тексты. Это навык быть
точным и вовремя поправлять».*

Этап 01 пройден — вы умеете формулировать
задачу, итерировать и откатываться.
Дальше — учим Claude помнить ваш проект
между сессиями.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 00
- Этап 02